Mobile Demo Script Explanation -- Justask (App Dev, Challenge 04)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: Use an explicit Intent for activities with no custom action, and an implicit Intent for ones that declare their own action.

Background -- what the terms below mean

What does "App Dev" mean, as a category? On this platform, **App Dev** challenges ask you to write your own small Android app that behaves a certain way. This is different from **Reversing** or **Exploitation** challenges, where you're given someone else's app to break into -- here there's usually no separate target app to download at all. You are writing code from scratch (or talking to a small server the challenge provides) to meet a spec.

What is an APK? The installable file for an Android app (like a .exe on Windows). For this category, the APK that matters is the one **you build and submit**, not a downloaded target.

What is logcat / the MOBISEC tag? Android apps can write messages to the phone's system log, called **logcat**. The grading harness watches for lines tagged **MOBISEC** and checks their content to decide whether to accept the submission.

How the APK was obtained

This one doesn't hand you a separate target APK to download at all. The challenge is to **write your own small app** that satisfies a spec (often by talking to a small server the platform runs, or to another running app component). The only piece kept from the target side is its own **AndroidManifest.xml**, used to see exactly how its components are declared. So the only APK in play here is the one **you** build and submit -- there's nothing to decompile.

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 04 Justask.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

The target app (com.mobisec.justask) has four activities, each holding one part of the flag, returned via the result Intent of a startActivityForResult() call. Per the target's own manifest: PartOne and PartThree are exported with no intent-filter action, so they must be started with an explicit Intent (component name); PartTwo and PartFour each declare their own custom action (JUSTASK and JUSTASKBUTNOTSOSIMPLE) and are started with an implicit Intent using that action. The four calls are chained through onActivityResult() and every extra key in each reply is dumped to logcat, since the exact key holding each flag fragment isn't specified.

Files to open

1. MainActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\App Dev/04 Justask\Android Studio Project\app\src\main\j
```

This is our own code -- open directly in Android Studio or any text editor.

2. Target AndroidManifest.xml:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\App Dev/04 Justask\Analysis and Scripts\Target AndroidMa

What to say -- explanation of the exploit, step by step

1. I obtained the target app's AndroidManifest.xml and identified its four activities and how each is addressed (explicit vs. implicit Intent).

2. I wrote MainActivity to call PartOne with an explicit Intent (component name), since it declares no custom action.

3. I called PartTwo with an implicit Intent using its declared action, com.mobisec.intent.action.JUSTASK.

4. I repeated the same pattern for PartThree (explicit) and PartFour (implicit, action JUSTASKBUTNOTSOSIMPLE), chaining each call through onActivityResult() so they run in sequence.

5. For every reply, I dumped all extras to logcat under the MOBISEC tag, since I didn't know in advance which key held each flag fragment.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log."

Result